DocLint

проверка оформления учебных работ по ГОСТ прямо в браузере

Отчёт по реализации проекта

Проект 3 · Колосов Егор Дмитриевич

2026

Локальный запуск: python3 -m http.server 8080

Суть проекта

DocLint — веб-приложение, которое за минуту проверяет курсовую, отчёт по практике или ВКР в формате .docx на соответствие ГОСТ 7.32-2017 или требованиям методички конкретной кафедры. Студент перетаскивает файл на страницу и получает список нарушений: в каком абзаце, что не так и как это исправить в Word. Приложение работает целиком в браузере: файл не загружается на сервер, регистрация не нужна. Сайт собирается как статический и подготовлен к публикации на GitHub Pages.

Гипотеза. Если студент сможет за одну минуту, без регистрации и без отправки файла в интернет проверить оформление работы по требованиям своей кафедры, он будет делать это перед каждой сдачей, а количество возвратов с нормоконтроля из-за оформления сократится.

Реализация. В проекте реализованы три функции MVP: проверка документа по профилю, понятный отчёт с привязкой к месту и профиль требований кафедры. Продукт собирается в статический сайт, работает без сервера и без сети после первого открытия. Проект покрыт 92 автоматическими тестами; проверка выполняется в браузере, документ не покидает компьютер.

Структура отчёта. Далее описаны постановка задачи и границы MVP, архитектура и поток данных, алгоритм вычисления оформления, реализация интерфейса и профилей, результаты проверки, ограничения и выводы.

1. Постановка задачи

1.1. Проблема

Перед каждой сдачей студент вручную сверяет оформление с ГОСТом и методичкой кафедры. Word не показывает, какие абзацы отличаются от требуемого оформления: чтобы увидеть шрифт, интервал или отступ, нужно выделить конкретный абзац и открыть его параметры. На курсовую в 25–40 страниц уходит от одного до трёх часов, а часть нарушений всё равно пропускается.

Требования кафедр расходятся с ГОСТом и друг с другом: кегль 12 или 14, правое поле 10 или 15 мм, разные правила для заголовков. Скопированный текст приносит «чужое» форматирование. Если работу возвращают с нормоконтроля, добавляется цикл «исправить → отправить → ждать ответа», который может стоить несколько дней перед защитой.

Существующие альтернативы закрывают проблему лишь частично: ИИ-сервисы требуют загрузки работы на чужой сервер и платны после пробного периода; шаблоны Word помогают только при работе в шаблоне с нуля; LaTeX требует навыков; руководитель и нормоконтролёр проверяют выборочно и медленно; фрилансеры стоят денег и времени.

Top-3 проблемы:

• **Критическая боль:** работу не принимают к защите, пока оформление не соответствует требованиям, а найти все нарушения вручную в документе на десятки страниц практически невозможно — что-то обязательно пропускается.

• **Значительная боль:** требования кафедр отличаются от ГОСТа и друг от друга, и студент не уверен, по какому набору правил сверяться.

• **Фоновая боль:** скопированный текст приносит чужой шрифт, интервал и отступы, которые внешне почти незаметны, но раздражают руководителя и нормоконтролёра.

1.2. Границы MVP

Вход — один файл .docx и профиль требований; выход — отчёт на странице. Ограничения интерфейса: один файл за раз, до 20 МБ; тип и размер проверяются до распаковки. Файл читается только в браузере, не отправляется на сервер и не сохраняется после проверки. Новый файл заменяет предыдущий отчёт; отчёт живёт до новой проверки или закрытия вкладки.

Ориентир производительности — документ на 40 страниц за 10 секунд. В MVP проверяются шесть параметров обычных абзацев основного текста;

титульный лист, заголовки, оглавление, таблицы и пустые абзацы пропускаются, чтобы не давать ложных срабатываний. Встроены два пресета, пользователь может создать свой профиль и обменяться им файлом .json.

2. Архитектура решения

2.1. Компонентный состав

Приложение построено как статический сайт без серверной части. HTML и CSS отвечают за интерфейс, JavaScript-модули — за приём файла, разбор OOXML, вычисление оформления, правила и отчёт. Модули вызываются напрямую из интерфейса и возвращают данные или типизированную ошибку; сетевых запросов, кроме загрузки статических файлов, нет.

Таблица 1 – Модули приложения

Модуль	**Ответственность**
index.html, app.js	Разметка страницы, состояния интерфейса, профили, загрузка, отчёт
file-reader.js	Проверка типа и размера, JSZip, конвейер проверки, прогресс
docx-parser.js	document.xml: порядок абзацев, текст, поля, исключения MVP
style-resolver.js	Наследование стилей, тема, символьные стили, единицы Word
rules.js	Сравнение с профилем, допуски, сборка Report
report.js	Категории, группировка соседних нарушений, сводка, фильтры
profiles.js, presets.js	Валидация профилей, LocalStorage, импорт и экспорт JSON
hints.js	Подписи категорий и подсказки с путями в меню Word
errors.js	Коды ошибок и единый формат {code, message, details}
demo.js	Демонстрационный .docx с намеренными ошибками — в памяти
sw.js	Service worker: офлайн-кэш статических файлов приложения

2.2. Поток данных

Проверка проходит по цепочке, в которой каждый следующий шаг использует результат предыдущего и не изменяет исходный документ:

1) пользователь выбирает профиль и файл .docx;

2) file-reader.js проверяет тип, размер и профиль;

3) JSZip извлекает word/document.xml, word/styles.xml и word/theme/theme1.xml, не трогая изображения;

4) docx-parser.js определяет порядок абзацев, их текст, исключения и поля страницы;

5) style-resolver.js вычисляет эффективное оформление каждого абзаца и фрагмента;

6) rules.js сравнивает значения с профилем, report.js группирует нарушения, интерфейс показывает сводку и карточки.

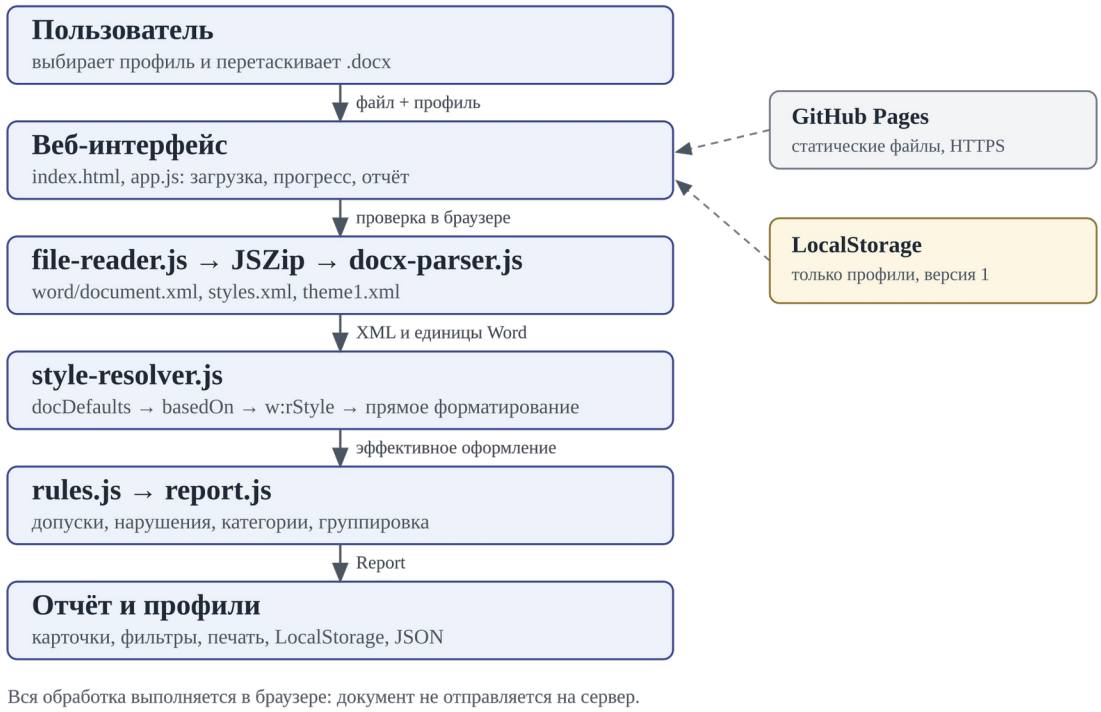


Рисунок 1 – Архитектура DocLint и поток проверки

2.3. Модель данных

Profile содержит id, название, версию схемы, поля страницы, шрифт, минимальный кегль, междустрочный интервал, абзацный отступ и выравнивание. Встроенные профили заданы в коде, пользовательские хранятся в LocalStorage под ключом doclint.profiles с версией схемы 1.

ParagraphRecord описывает абзац: индекс, превью первых слов, вычисленные свойства, номер раздела и причину пропуска. **Violation** содержит индекс абзаца, превью, правило, фактическое и требуемое значения и подсказку. **Report** объединяет число проверенных и пропущенных абзацев, счётчики по правилам, сводку и список нарушений.

Инварианты: у профиля уникальный id, встроенные профили нельзя перезаписать; у нарушения ключ — пара (paragraphIndex, ruleId); все поля профиля обязательны, числа конечны и положительны, кроме допустимого нулевого отступа; неизвестные поля JSON отбрасываются.

3. Алгоритм проверки

3.1. Разбор OOXML и единицы Word

Файл .docx — это ZIP-архив с XML-частями. Приложение читает только три файла: word/document.xml (порядок и содержимое абзацев), word/styles.xml (стили и docDefaults) и word/theme/theme1.xml (шрифты темы). Изображения и другие части не распаковываются, поэтому размер документа почти не влияет на время проверки.

Единицы Word переводятся аккуратно: поля и отступы хранятся в twips (1/20 пункта, 1440 twips = 1 дюйм), размер шрифта — в половинах пункта, интервал — в 1/240 строки. Для каждого значения предусмотрен допуск на округление, чтобы 30 мм не превращались в ложное нарушение из-за пересчёта.

3.2. Наследование оформления

Итоговое оформление абзаца складывается из нескольких уровней. Приложение вычисляет его в порядке: **docDefaults → цепочка basedOn стиля абзаца → символьный стиль w:rStyle → прямое форматирование абзаца → прямое форматирование фрагмента (run)**. Шрифт может быть задан ссылкой на тему документа; тогда значение берётся из theme1.xml, включая кириллический вариант шрифта.

Если стиль отсутствует, цепочка basedOn зациклена или значение определить нельзя, абзац или правило помечаются как непроверенные. Это консервативное решение: лучше пропустить проверку, чем показать ошибку там, где её нет.

3.3. Исключения и допуски

В MVP пропускаются титульный лист, заголовки, оглавление, таблицы, пустые абзацы и абзацы с неизвестным стилем. Граница титульного листа определяется по первому явному разрыву страницы, разрыву раздела или свойству «с новой страницы». Если разрывов нет вообще, границу найти нельзя, поэтому проверяется весь документ — иначе короткие работы без разрыва давали бы пустой отчёт.

Таблица 2 – Проверяемые параметры и допуски

Параметр	**Допуск**	**Поведение при неизвестном значении**
Поля страницы (4 стороны)	±0,5 мм	правило пропускается
Шрифт	точное совпадение	правило пропускается
Размер шрифта	±0,5 пт, не меньше профиля	правило пропускается
Междустрочный интервал	±0,05	exact/atLeast пропускается
Абзацный отступ	±0,5 мм	firstLineChars пропускается
Выравнивание	точное совпадение	неизвестное значение пропускается

Нарушения сортируются по месту в документе. Одинаковые нарушения с одним правилом и одним фактическим значением в соседних абзацах объединяются в одну карточку (например, «Абзацы 12–51: шрифт Calibri»). Нарушения полей страницы остаются отдельными карточками, потому что относятся к странице, а не к абзацу.

3.4. Отчёт

Отчёт состоит из сводки, фильтров по категориям и карточек. В карточке показаны место (абзац или диапазон), первые слова абзаца, категория, найденное и требуемое значения и подсказка с путём в меню Word. Пользователь может отфильтровать нарушения по категориям и распечатать отчёт или сохранить его в PDF стандартной печатью браузера.

Если нарушений нет, отчёт сообщает об отсутствии нарушений именно в проверяемых правилах и отдельно указывает число проверенных и пропущенных абзацев. Это важно: «чистый» отчёт не означает, что документ полностью соответствует всем требованиям кафедры за пределами MVP.

4. Реализация

4.1. Интерфейс и состояния

Страница адаптивная и состоит из четырёх областей: выбор профиля, зона загрузки, отчёт и форма профиля. Реализованы четыре состояния: пустое (выбрать файл или открыть пример), загрузка (индикатор и этапы проверки, повторный запуск недоступен), успех (сводка, проверенные и пропущенные абзацы) и ошибка (причина и действие — выбрать другой файл, уменьшить размер или исправить JSON профиля).

Интерфейс поддерживает перетаскивание файла, выбор кнопкой, клавиатурный фокус, адаптивную вёрстку до 320 пикселей и печатные стили. Кнопка «Проверить пример» собирает демонстрационный документ в памяти, поэтому работает без сети. Service worker кэширует статические файлы, и после первого открытия приложение доступно офлайн, включая перезагрузку вкладки.

4.2. Профили требований

Встроены два пресета: «ГОСТ 7.32-2017» (поля 30/15/20/20 мм, Times New Roman, кегль не меньше 12 пт, интервал 1,5, отступ 12,5 мм, по ширине) и «Типовая методичка (кегль 14)» с тем же набором параметров и минимальным кеглем 14 пт.

Свой профиль создаётся формой: название, поля, шрифт, кегль, интервал, отступ и выравнивание. Профиль сохраняется в LocalStorage, выгружается в .json и импортируется у одногруппников. Импорт проверяет все поля и отбрасывает неизвестные; встроенные профили перезаписать нельзя — при импорте с их id создаётся копия с новым id.

4.3. Ошибки и ограничения ввода

Ошибки имеют единый формат {code, message, details} и понятное сообщение в интерфейсе. Коды: INVALID_FILE, FILE_TOO_LARGE, INVALID_DOCX, INVALID_PROFILE и CHECK_FAILED. Старый

формат .doc отклоняется с предложением сохранить файл как .docx; повреждённый ZIP или отсутствие word/document.xml дают сообщение без падения страницы; файл больше 20 МБ отклоняется до распаковки. После ошибки пользователь может сразу выбрать другой файл.

4.4. Тесты и непрерывная интеграция

Проект покрыт 92 автоматическими тестами: перевод единиц, разбор XML, наследование стилей и темы, символьные стили, исключения MVP, правила с допусками, группировка отчёта, профили и LocalStorage, импорт и экспорт JSON, сквозные проверки реальных .docx. Тесты запускаются на Node.js через node --test и на Bun.

Дополнительно проверены: демонстрационный документ (21 нарушение в 6 категориях), документ без нарушений, мобильная вёрстка 375 пикселей, печатные стили, офлайн-режим после перезагрузки и файл, созданный в LibreOffice. На документе из 1200 абзацев проверка занимает около 0,13 секунды — с большим запасом относительно ориентира 10 секунд.

GitHub Actions на каждый push запускает тесты. Собранный статический сайт формируется скриптом tools/publish-site.sh и подготовлен к публикации на GitHub Pages. В публикацию попадают только статические файлы приложения, без тестов и служебных данных.

5. Результаты

5.1. Проверка продукта

Production-сборка проверена локально: статический сервер отдаёт страницу, «Проверить пример» строит отчёт, service worker включает офлайн-режим. Для проверки преподавателем достаточно запустить python3 -m http.server 8080 в каталоге проекта и открыть http://localhost:8080; сервер, учётные записи и установка не нужны.

• демонстрационный документ: 21 нарушение в 6 категориях, 11 карточек, включая сгруппированные соседние абзацы;

• корректный документ: отчёт пустой, показано число проверенных абзацев и пояснение границ MVP;

• профиль методички с кеглем 14 добавляет нарушение кегля в том же документе;

• мобильная вёрстка 375 пикселей без горизонтального переполнения;

• печать скрывает интерфейс и оставляет отчёт с карточками;

• офлайн после первого открытия: страница и проверка примера работают без сети;

• ошибки .doc, повреждённого ZIP, файла без document.xml и неверного JSON показываются понятным сообщением и не скрывают возможность продолжить работу.

5.2. Ограничения

Ограничения зафиксированы в интерфейсе, README и инструкции пользователя. Они не относятся к трём функциям MVP, но важны для честной оценки продукта.

Таблица 3 – Ограничения текущей версии

Ограничение	**Причина**	**Что делать**
Поля проверяются по финальному разделу	MVP работает с одним набором полей	проверить разделы вручную
Абзацы списков проверяются как обычные	списки не входят в исключения MVP	учитывать выступ вручную
exact/atLeast и firstLineChars пропускаются	значение нельзя надёжно перевести в кратность/мм	проверить в Word
Отслеживаемые изменения не анализируются	нужен отдельный разбор w:ins/w:del	принять исправления
Титульный лист без разрыва неотличим от текста	нет явной границы страницы	добавить разрыв или проверить вручную
Нет проверки на реальных Word/Google Docs	этих файлов не было в среде разработки	прогнать на своих файлах

5.3. Риски и митигации

Ложные срабатывания. Если DocLint покажет ошибку там, где её нет, пользователь перестанет доверять отчёту. Митигация: консервативные правила, показ найденного и требуемого значения, пропуск спорных элементов, счётчик непроверенных правил и тесты наследования стилей.

Требования кафедры шире MVP. Пользователь может получить «чистый» отчёт, но работу вернут из-за подписей, списка литературы или титульного листа. Митигация: на странице явно перечислено, что проверяется и что нет; расширенные проверки — первые кандидаты в следующую версию.

Недоверие к загрузке работы. Студенты могут бояться утечки. Митигация: обработка только в браузере, отсутствие сервера и аналитики, открытый исходный код, явное сообщение «файл не покидает компьютер» и работа без сети после загрузки.

6. Выводы

Цель проекта достигнута: реализован рабочий MVP и проверена его production-сборка. DocLint проверяет оформление .docx по ГОСТу или методичке кафедры прямо в браузере. Три заявленные функции — проверка, отчёт и профиль — работают end-to-end; документ не отправляется на сервер, а сайт не требует поддержки серверной инфраструктуры.

Архитектура выдержана в границах статического приложения: JSZip и DOMParser разбирают OOXML, модуль стилей вычисляет итоговое оформление с учётом наследования, правила применяют допуски, отчёт группирует нарушения. Тесты автоматизированы, публикация выполняется одной командой; проверка 1200 абзацев занимает около 0,13 секунды, что превышает требование производительности с запасом.

Следующие шаги: проверка полей по разделам, корректная обработка списков, расширенные правила для заголовков, подписей, списка литературы и титульного листа, а также прогон на файлах из Word и Google Docs. Эти

улучшения не меняют архитектуру и могут быть добавлены поверх текущего ядра.

Приложение А. Запуск и проверка

Локальный запуск и проверка выполняются командами:

• npm test — запуск 92 тестов на Node.js; bun test — альтернативный запуск на Bun;

• python3 -m http.server 8080 — локальный статический сервер, затем http://localhost:8080;

• bun tools/make-sample.mjs — пересборка sample/demo.docx;

• node tools/make-sample.mjs — та же сборка без Bun.

Исходный код: https://github.com/BombasterSirni/DocLint. Инструкция пользователя: docs/user-guide.md.